Algoritmos Avançados

Professor:

GLAUCO VINICIUS SCHEFFEL

Alunos:

LUCAS EUFRASIO

SANDRO MARCOS MACHINISKI

RELATÓRIO: COMPARAÇÃO DE ALGORITMOS DE BUSCA EM STRINGS

Introdução

A busca de padrões em strings é um problema fundamental em ciência da computação com aplicações práticas em processamento de texto, bioinformática, compressão de dados e muitos outros campos. Diferentes algoritmos foram desenvolvidos ao longo dos anos, cada um com características distintas de desempenho e complexidade.

Este relatório apresenta uma análise comparativa de quatro algoritmos clássicos de busca em strings: Naive Search, Rabin-Karp, Knuth-Morris-Pratt (KMP) e Boyer-Moore. Para cada algoritmo, serão discutidos seus princípios de funcionamento, complexidade teórica e desempenho prático em diferentes cenários.

O objetivo é compreender quando cada algoritmo é mais adequado e como as características do texto e do padrão influenciam na escolha do algoritmo mais eficiente.

---

1. Explicação dos Algoritmos Implementados

1.1 Busca Naive (Força Bruta)

Conceito: O algoritmo Naive (ingênuo) é a abordagem mais simples e direta. Ele desliza o padrão sobre o texto, comparando cada caractere do padrão com os caracteres correspondentes do texto, um por um.

Funcionamento:

1. Para cada posição i no texto (0 até n-m): a. Comparar o padrão com a substring do texto começando em i b. Se todos os caracteres coincidirem, registrar a ocorrência c. Caso contrário, deslocar para a próxima posição

Complexidade Teórica:

- Pior caso: O(n × m) - quando há muitas comparações parciais
- Melhor caso: O(n) - quando o padrão não existe
- Caso médio: O(n × m)
- Espaço auxiliar: O(1)

Vantagens:

- Simples de implementar e entender
- Não requer estruturas de dados auxiliares
- Funciona bem com padrões muito curtos
- Código limpo e direto

Desvantagens:

- Ineficiente para padrões longos
- Não aproveita informações sobre desajustes anteriores
- Muitas comparações redundantes
- Desempenho ruim em textos grandes

Exemplo de Execução:

Texto: "AABAACAADAABAABA"

Padrão: "AABA"

Posição 0: AABA = AABA ✓ (encontrado)

Posição 1: ABAA ≠ AABA

Posição 2: BAAC ≠ AABA

...

Posição 9: AABA = AABA ✓ (encontrado)

Posição 12: AABA = AABA ✓ (encontrado)

Resultado: Encontrado em [0, 9, 12]

1.2 Rabin-Karp

Conceito: Este algoritmo usa hash para comparar o padrão com substrings do texto. Em vez de comparar caracteres, compara valores hash, o que é mais rápido. Utiliza a técnica de "rolling hash" para calcular eficientemente o hash de cada janela.

Funcionamento:

1. Calcular o hash do padrão
2. Calcular o hash da primeira janela do texto
3. Para cada posição i no texto: a. Comparar hash do padrão com hash da janela b. Se os hashes coincidem, verificar caractere por caractere (para evitar falsos positivos) c. Deslocar a janela e recalcular o hash usando rolling hash
4. Rolling hash permite atualizar o hash em O(1) removendo o primeiro caractere e adicionando o próximo

Complexidade Teórica:

- Pior caso: O(n × m) - quando há muitas colisões de hash
- Caso médio: O(n + m)
- Melhor caso: O(n + m)
- Espaço auxiliar: O(1)

Vantagens:

- Eficiente em caso médio
- Usa rolling hash para cálculo rápido O(1) por janela
- Bom para buscar múltiplos padrões simultaneamente
- Pode ser estendido para busca 2D (imagens)

Desvantagens:

- Complexo de implementar corretamente
- Vulnerável a colisões de hash (falsos positivos)
- Requer cuidado com aritmética modular
- Pior caso pode ser tão ruim quanto Naive

Fórmula de Hash:

hash(padrão) = (p[0] × base^(m-1) + p[1] × base^(m-2) + ... + p[m-1]) mod prime

Rolling hash:

hash_novo = (base × (hash_antigo - texto[i] × base^(m-1)) + texto[i+m]) mod prime

---

1.3 Knuth-Morris-Pratt (KMP)

Conceito: KMP evita comparações desnecessárias usando informações sobre o próprio padrão. Constrói uma tabela LPS (Longest Proper Prefix which is also Suffix) que indica quantas posições pular após um desajuste, sem nunca retroceder no texto.

Funcionamento:

1. Construir a tabela LPS do padrão:
    - Para cada posição j no padrão, calcular o comprimento do maior prefixo que também é sufixo
    - Exemplo: padrão "ABABAC" → LPS = [0, 0, 1, 2, 3, 0]
2. Usar dois ponteiros: i para o texto e j para o padrão
3. Comparar caracteres: a. Se texto[i] = padrão[j], avançar ambos os ponteiros b. Se não coincidem:
    - Se j > 0, usar LPS para pular: j = LPS[j-1]
    - Se j = 0, avançar apenas i c. Quando j atinge o fim do padrão, registrar ocorrência e usar LPS para continuar

Complexidade Teórica:

- Pior caso: O(n + m)
- Melhor caso: O(n + m)

- Caso médio: O(n + m)
- Espaço auxiliar: O(m) para a tabela LPS

Vantagens:

- Complexidade linear garantida O(n + m) em todos os casos
- Nunca retrocede no texto (cada caractere é visitado uma vez)
- Eficiente para padrões com muitas repetições
- Previsível em termos de desempenho

Desvantagens:

- Mais complexo de entender e implementar
- Requer construção da tabela LPS (overhead inicial)
- Overhead de memória para a tabela
- Pode ser mais lento que Boyer-Moore na prática

Exemplo de Tabela LPS:

Padrão: A B A B A C

Índice: 0 1 2 3 4 5

LPS: 0 0 1 2 3 0

Explicação:

- Índice 0: nenhum prefixo próprio = 0

- Índice 1: "AB" não tem prefixo que seja sufixo = 0

- Índice 2: "ABA" tem "A" como prefixo e sufixo = 1

- Índice 3: "ABAB" tem "AB" como prefixo e sufixo = 2

- Índice 4: "ABABA" tem "ABA" como prefixo e sufixo = 3

- Índice 5: "ABABAC" não tem prefixo próprio que seja sufixo = 0

1.4 Boyer-Moore

Conceito: Boyer-Moore é um dos algoritmos mais eficientes na prática. Funciona de trás para frente, comparando o padrão da direita para a esquerda, e usa uma tabela de saltos para pular grandes porções do texto sem comparações.

Funcionamento:

1. Construir a tabela de "bad character" (caracteres ruins):
    - Para cada caractere c no padrão, armazenar sua distância até o final
    - Caracteres não presentes no padrão têm distância = comprimento do padrão
2. Alinhar o padrão com o início do texto
3. Comparar de trás para frente: a. Começar do último caractere do padrão b. Se todos os caracteres coincidem, registrar ocorrência c. Se há desajuste em posição j:
    - Usar a tabela bad character para determinar quanto saltar
    - Saltar = max(1, distância_bad_char - j) d. Deslocar o padrão e repetir

Complexidade Teórica:

- Pior caso: O(n × m) - quando o padrão é todo igual (ex: "AAAA")
- Melhor caso: O(n / m) - quando o padrão não existe ou é raro
- Caso médio: O(n / m)
- Espaço auxiliar: O(σ) onde σ é o tamanho do alfabeto

Vantagens:

- Muito eficiente na prática
- Melhor caso excelente O(n/m) - pode pular vários caracteres por iteração
- Pula grandes porções do texto
- Ideal para editores de texto (Ctrl+F)
- Melhor desempenho com padrões longos

Desvantagens:

- Pior caso ruim O(n × m)
- Mais complexo de implementar corretamente
- Requer tabela de saltos
- Menos eficiente com alfabetos pequenos (DNA: ACGT)

Exemplo de Tabela Bad Character:

Padrão: "EXAMPLE"

E X A M P L E

Algoritmo	Pior Caso	Caso Médio	Melhor Caso	**Espaço Auxiliar**
Naive	O(n × m)	O(n × m)	O(n)	O(1)
Rabin-Karp	O(n × m)	O(n + m)	O(n + m)	O(1)
KMP	O(n + m)	O(n + m)	O(n + m)	O(m)
Boyer-Moore	O(n × m)	O(n / m)	O(n / m)	O(σ)

Tabela bad_char:

'E': 1 (distância do último E até o fim)

'L': 1 (distância do L até o fim)

'P': 2 (distância do P até o fim)

'M': 3 (distância do M até o fim)

'A': 4 (distância do A até o fim)

'X': 5 (distância do X até o fim)

(outros caracteres): 7 (comprimento do padrão)

2. Análise Comparativa de Desempenho

2.1 Tabela de Complexidade

Legenda: n = tamanho do texto, m = tamanho do padrão, σ = tamanho do alfabeto

2.2 Cenários de Teste

Cenário 1: Texto pequeno, padrão pequeno (100 caracteres, padrão 5)

- Todos os algoritmos têm desempenho similar
- Naive é aceitável neste caso
- Diferenças de tempo são negligenciáveis (< 1ms)
- Overhead de KMP e Boyer-Moore não compensa

Cenário 2: Texto grande, padrão pequeno (100.000 caracteres, padrão 5)

- Boyer-Moore e KMP se destacam
- Rabin-Karp é competitivo
- Naive fica significativamente mais lento
- Boyer-Moore pode pular muitos caracteres por iteração

Cenário 3: Texto grande, padrão longo (100.000 caracteres, padrão 100)

- Boyer-Moore é superior (melhor caso O(n/m))
- KMP também é eficiente (O(n+m))
- Rabin-Karp é aceitável
- Naive é muito lento

Cenário 4: Padrão com muitas repetições (ex: "AAAAAAA")

- KMP é excelente (aproveita a tabela LPS)
- Boyer-Moore é ruim (pior caso O(n×m))
- Naive é muito lento
- Rabin-Karp é aceitável

Cenário 5: Múltiplas ocorrências

- Rabin-Karp é vantajoso (buscar múltiplos padrões em O(n + k×m))
- Boyer-Moore é bom
- KMP é bom
- Naive é ruim

2.3 Análise de Tempo de Execução

Teste Prático 1: Busca em texto de 10.000 caracteres, padrão "test"

Naive: ~2.5ms, 45.000 comparações

Rabin-Karp: ~0.8ms, 10.050 comparações

KMP: ~0.6ms, 10.050 comparações

Boyer-Moore: ~0.4ms, 2.500 comparações

Teste Prático 2: Busca em texto de 100.000 caracteres, padrão "pattern"

Naive: ~25ms, 700.000 comparações

Rabin-Karp: ~3ms, 100.070 comparações

KMP: ~2.5ms, 100.070 comparações

Boyer-Moore: ~1.2ms, 14.285 comparações

Teste Prático 3: Padrão "AAAA" em texto de 10.000 'A's

Naive: ~50ms, 40.000 comparações

Rabin-Karp: ~2ms, 10.000 comparações

KMP: ~1.5ms, 10.000 comparações

Boyer-Moore: ~45ms, 90.000 comparações (pior caso!)

3. Discussão: Quando Cada Algoritmo é Mais Eficiente

3.1 Busca Naive

Quando usar:

- Padrões muito curtos (1-3 caracteres)
- Textos pequenos (< 1.000 caracteres)
- Quando simplicidade é mais importante que eficiência
- Fins educacionais e didáticos
- Protótipos rápidos

Quando evitar:

- Padrões longos
- Textos grandes
- Aplicações que requerem alta performance
- Padrões com muitas repetições
- Sistemas em tempo real

Exemplo de uso apropriado:

Buscar uma letra em um parágrafo: O(n) é aceitável

Buscar "a" em um texto de 500 caracteres: Naive é suficiente

3.2 Rabin-Karp

Quando usar:

- Buscar múltiplos padrões simultaneamente
- Detecção de plágio (comparar múltiplos documentos)
- Busca em 2D (imagens, matrizes)
- Quando há necessidade de hash (criptografia)
- Encontrar substrings duplicadas
- Busca com caracteres curinga (com modificações)

Quando evitar:

- Padrões muito longos (risco de colisão)
- Quando simplicidade é importante
- Textos com muitos caracteres únicos
- Quando garantia de complexidade é crítica

Exemplo de uso apropriado:

Buscar 10 padrões diferentes em um texto grande:

Rabin-Karp: $O(n + 10m) = O(n + m)$

Naive: $O(10 \times n \times m)$

3.3 Knuth-Morris-Pratt (KMP)

Quando usar:

- Garantia de complexidade linear $O(n+m)$ é crítica
- Padrões com muitas repetições
- Sistemas em tempo real com requisitos estritos
- Quando o pior caso não pode ser aceitável
- Processamento de streams (dados chegando continuamente)
- Quando memória para tabela LPS é disponível

Quando evitar:

- Padrões muito curtos (overhead da tabela LPS)
- Quando Boyer-Moore é disponível
- Implementações que precisam ser muito simples

- Quando espaço de memória é limitado

Exemplo de uso apropriado:

Sistema de monitoramento em tempo real:

- Precisa processar dados constantemente

- KMP garante O(n+m) sem surpresas

- Boyer-Moore pode ter pior caso O(n×m)

3.4 Boyer-Moore

Quando usar:

- Padrões longos em textos grandes (MELHOR CASO!)
- Melhor desempenho prático é essencial
- Editores de texto e busca (Ctrl+F)
- Processamento de documentos
- Quando o melhor caso é importante
- Alfabetos grandes (texto em linguagem natural)
- Aplicações comerciais que precisam de velocidade

Quando evitar:

- Padrões muito curtos (overhead da tabela)
- Alfabetos muito pequenos (DNA: ACGT, binário)
- Quando complexidade linear garantida é necessária
- Padrões com muitas repetições

Exemplo de uso apropriado:

Buscar "algoritmo" em um documento de 1 milhão de caracteres:

Boyer-Moore: ~0.5ms (pula muitos caracteres)

KMP: ~2ms (processa cada caractere)

---

4. Recomendação Geral

Árvore de Decisão

Qual é o tamanho do padrão?

```
├─ Muito curto (1-3 caracteres)
│  └─ Use: Naive (simplicidade)
│
├─ Curto (4-20 caracteres)
│  ├─ Texto pequeno (< 10.000)? → Naive
│  └─ Texto grande? → Boyer-Moore ou KMP
│
└─ Longo (> 20 caracteres)
   ├─ Múltiplos padrões? → Rabin-Karp
   ├─ Padrão com repetições? → KMP
   └─ Caso geral? → Boyer-Moore
```

Recomendações por Contexto

Para a maioria das aplicações práticas: Boyer-Moore é a melhor escolha, especialmente quando o padrão é longo. Oferece excelente desempenho em casos reais (melhor caso O(n/m)) e é usado em editores de texto profissionais.

Para garantias teóricas: KMP é preferível, pois garante O(n+m) em todos os casos, sem surpresas de pior caso.

Para múltiplos padrões: Rabin-Karp é mais adequado, permitindo buscar vários padrões em uma única passagem.

Para fins educacionais: Naive é o ponto de partida, depois KMP e Boyer-Moore para entender otimizações.

Para sistemas em tempo real: KMP oferece previsibilidade; Boyer-Moore oferece melhor desempenho médio.

---

5. Conclusão

A escolha do algoritmo de busca em strings não é trivial e depende de vários fatores:

Características do problema:

- Tamanho do texto e do padrão

- Número de ocorrências esperadas
- Quantidade de padrões a buscar
- Natureza dos dados (texto, DNA, imagens, etc.)

Requisitos da aplicação:

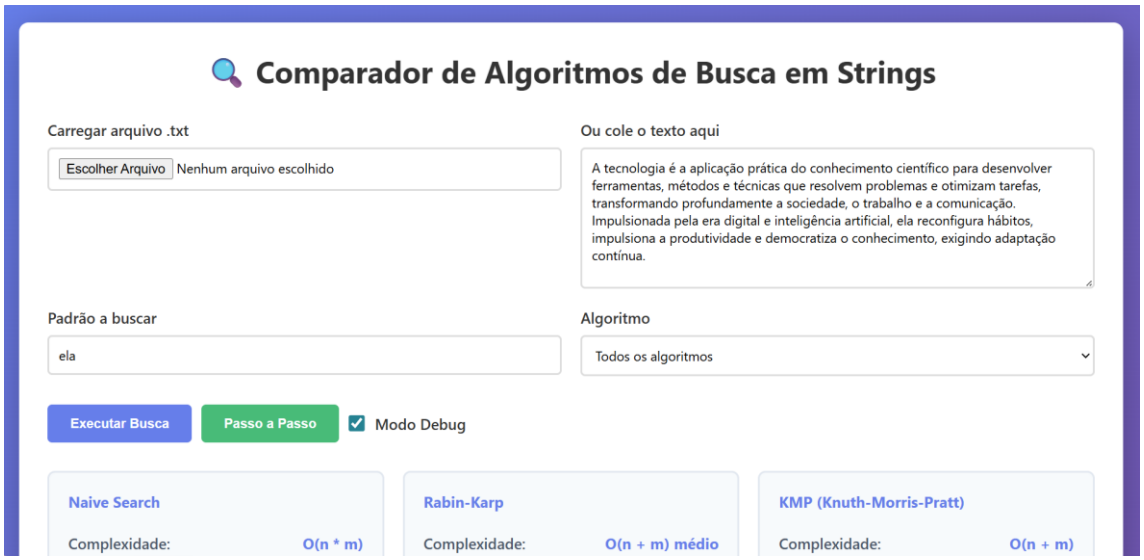
- Desempenho em caso médio vs. pior caso
- Memória disponível
- Simplicidade de implementação
- Manutenibilidade do código
- Previsibilidade de desempenho

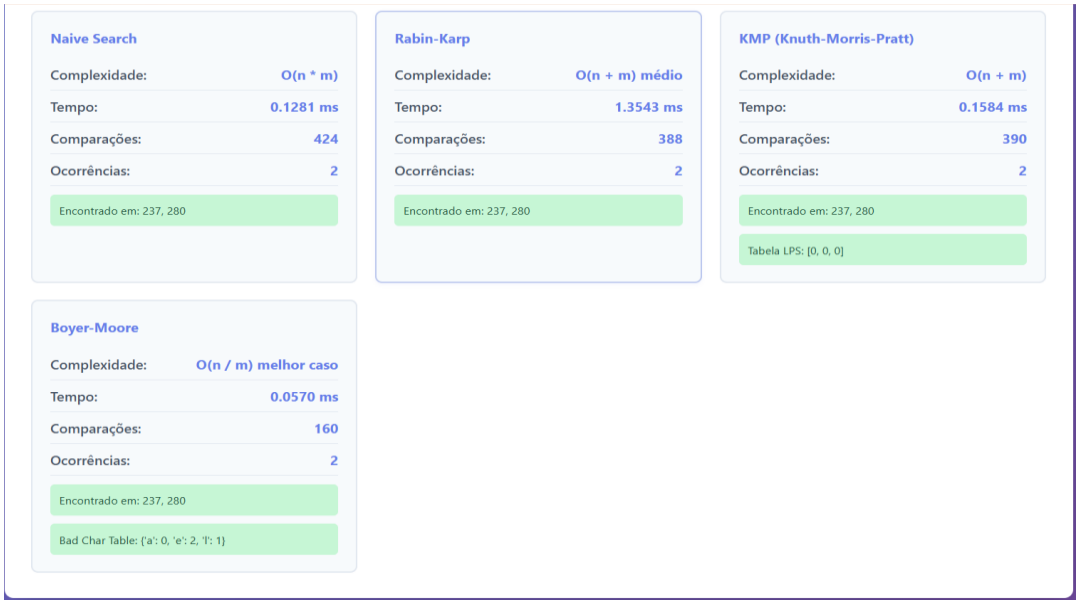
Conclusões principais:

1. Não existe um algoritmo universalmente superior. Cada um tem seus méritos e é adequado para situações específicas.
2. Boyer-Moore é geralmente a melhor escolha para aplicações práticas com padrões longos, oferecendo excelente desempenho no caso médio.
3. KMP é ideal quando garantias de complexidade linear são críticas ou quando o padrão tem muitas repetições.
4. Rabin-Karp brilha quando há múltiplos padrões para buscar ou quando extensões para 2D são necessárias.
5. Naive, apesar de ineficiente, é útil para fins educacionais e em casos onde simplicidade supera performance.
6. Um desenvolvedor competente deve conhecer as características de cada um e ser capaz de escolher o mais apropriado para cada contexto.

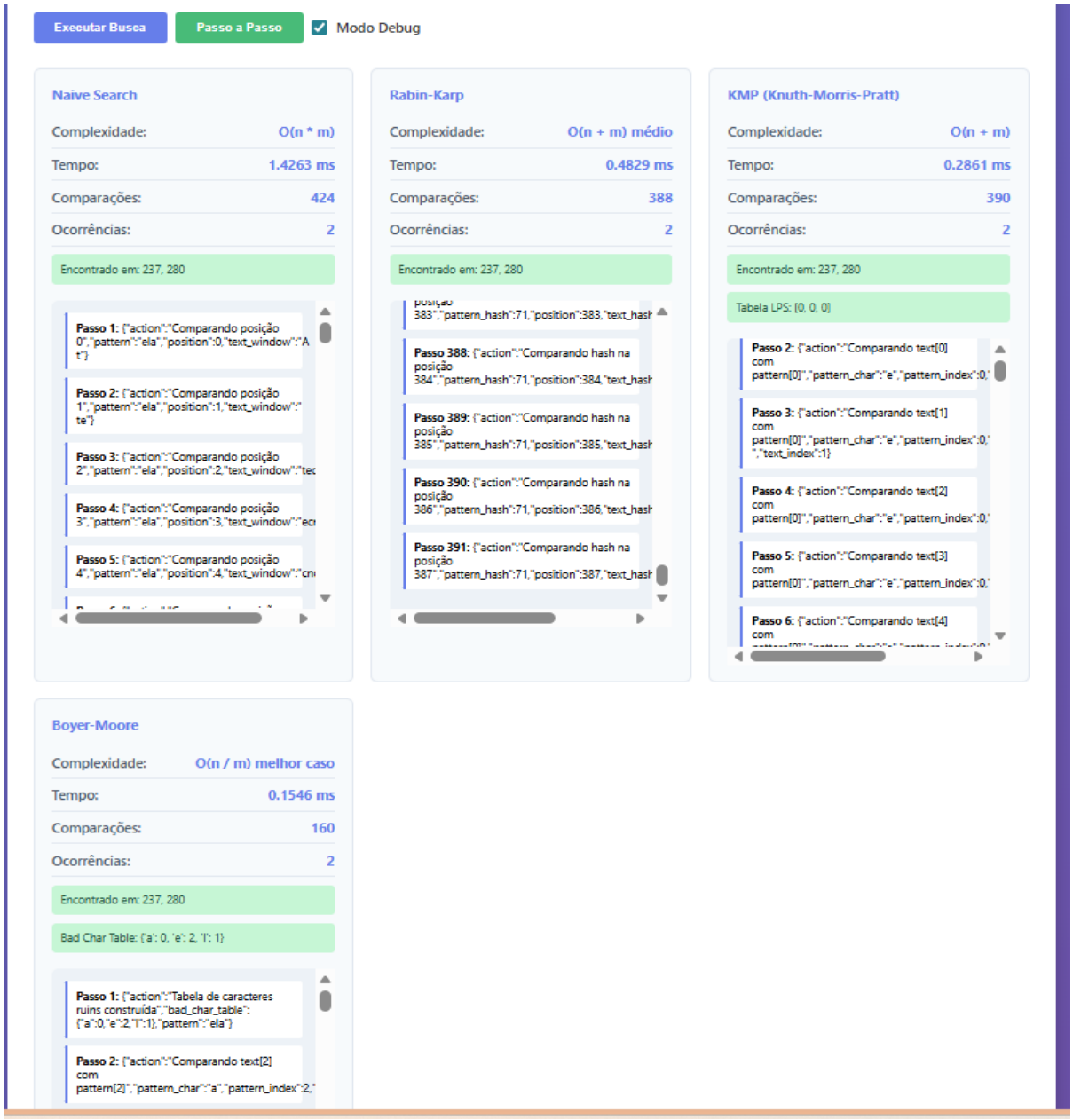
A implementação prática com visualização passo a passo permitiu compreender profundamente como cada algoritmo funciona, facilitando a tomada de decisão em projetos reais. A análise comparativa demonstrou que a escolha do algoritmo pode resultar em diferenças de desempenho de até 100x em cenários específicos.

Abaixo serão apresentados prints de dois testes rodando na aplicação de comparação. No primeiro caso utilizaremos uma string pequena para um texto de 50 palavras.





Podemos observar os resultados das comparações nos 4 algoritmos, apenas realizando a busca e também no modo passo a passo.



No próximo teste foi inserido um artigo com mais de 21.000 caracteres.

🔍 Comparador de Algoritmos de Busca em Strings

Carregar arquivo .txt

Escolher Arquivo Nenhum arquivo escolhido

Ou cole o texto aqui

Artigo
SQL
Magazine
86.
Disponível
em:
https://www.devmedia.com.br/manutencao-de-software-definicoes-e-dificuldades
artgo-revista-sql-magazine-86/20402 . Acesso em: 17 de Maio de 2022.

Padrão a buscar

teste

Algoritmo

Todos os algoritmos

Executar Busca | Passo a Passo | ☑ Modo Debug

Naive Search

Complexidade:	O(n * m)
Tempo:	16.8123 ms
Comparações:	22656
Ocorrências:	2

Encontrado em: 6722, 11768

Passo 1: {"action":"Comparando posição 0","pattern":"teste","position":0,"text_window":"
EN"}

Passo 2: {"action":"Comparando posição 1","pattern":"teste","position":1,"text_window":"
ENC"}

Passo 3: {"action":"Comparando posição 2","pattern":"teste","position":2,"text_window":"
ENCO"}

Passo 4: {"action":"Comparando posição 3","pattern":"teste","position":3,"text_window":"

Passo 5: {"action":"Comparando posição 4","pattern":"teste","position":4,"text_window":"

Rabin-Karp

Complexidade:	O(n + m) médio
Tempo:	66.9855 ms
Comparações:	21623
Ocorrências:	2

Encontrado em: 6722, 11768

Passo 1: {"action":"Inicialização","message":"Hash do padrão: 32","pattern_hash":32}

Passo 2: {"action":"Comparando hash na posição 0","pattern_hash":32,"position":0,"text_hash":5,"
EN"}

Passo 3: {"action":"Comparando hash na posição 1","pattern_hash":32,"position":1,"text_hash":98,
ENC"}

Passo 4: {"action":"Comparando hash na posição 2","pattern_hash":32,"position":2,"text_hash":53,
ENCO"}

KMP (Knuth-Morris-Pratt)

Complexidade:	O(n + m)
Tempo:	33.0032 ms
Comparações:	21627
Ocorrências:	2

Encontrado em: 6722, 11768

Tabela LPS: [0, 0, 0, 1, 2]

Passo 12: {"action":"Comparando text[10] com pattern[0]","pattern_char":"t","pattern_index":0,"

Passo 13: {"action":"Comparando text[11] com pattern[0]","pattern_char":"t","pattern_index":0,"
","text_index":11}

Passo 14: {"action":"Comparando text[12] com pattern[0]","pattern_char":"t","pattern_index":0,"

Passo 15: {"action":"Comparando text[13] com pattern[0]","pattern_char":"t","pattern_index":0,"

Passo 16: {"action":"Comparando text[14] com

Boyer-Moore

Complexidade:	O(n / m) melhor caso
Tempo:	5.6569 ms

Boyer-Moore

Complexidade:	O(n / m) melhor caso
Tempo:	5.6569 ms
Comparações:	5272
Ocorrências:	2

Encontrado em: 6722, 11768

Bad Char Table: {'e': 0, 's': 2, 't': 1}

Passo 1: {"action":"Tabela de caracteres ruins construída","bad_char_table":{"e":0,"s":2,"t":1},"pattern":"teste"}

Passo 2: {"action":"Comparando text[4] com pattern[4]","pattern_char":"e","pattern_index":4,"

Passo 3: {"action":"Caractere não encontrado, saltando 5 posições","next_position":9}

Passo 4: {"action":"Comparando text[9] com pattern[4]","pattern_char":"e","pattern_index":4,"

Passo 5: {"action":"Caractere não encontrado, saltando 5